

1 and 2.

```
divPoly (divisor, dividend) {
    if dividend == NaN OR divisor == NaN OR divisor == 0 {
        return NaN;
    }
    else if dividend degree < divisor degree OR dividend == 0 {
        return 0;
    }
    else {
        quotients[dividend degree + 1 - divisor degree];
        remainder[dividend degree] = dividend.coefs.copy();
        int k = dividend degree - divisor degree;
        int j = 1;
        while remainder degree >= divisor degree AND remainder degree >= 0 AND k >= 0 {
            t = remainder[remainder degree - j] / divisor[divisor degree - 1];
            quotient[k] = t;
            j = j + 1;
            k = k - 1;
            for i between (remainder degree - divisor - j + 2) and (remainder degree - j + 2) {
                Remainder[i] = remainder[i] - (t * divisor[divisor degree + i + j - remainder degree - 2]);
            }
        }
        return quotients;
    }
}
```

3. Loop invariant: remainder degree $\geq$ divisor degree $\wedge$ remainder degree $\geq$ 0

BASE CASE: does the LI hold before the loop?

remainder degree $\geq$ divisor degree

the LI holds, given that dividend.coefs $\geq$ divisor degree $\wedge$ remainder[dividend degree] = dividend.coefs.copy() $\wedge$ remainder degree $\geq$ 0 (from if statements) //by precondition

INDUCTION: assume the LI holds at iteration k, does it hold at iteration k+1?

- The degree of both the divisor and the remainder never changes, so therefore the LI must hold at the next iteration
- If remainder degree < divisor degree $\vee$ remainder degree < 0 at iteration k, there would be no iteration k+1

Does the loop terminate?

- Define D == k - 1 // initially k == divisor degree - dividend degree and k $\geq$ 0, D $\geq$ 0
- D decreases by 1 with each step.
- D eventually reaches 0.

- $D == 0 \Rightarrow$ loop exit condition $k < 0$
- When $D == 0$, loop exits